Orbit Wars — v12 Problem Setup, Representation, Reward & League

The shared foundation: the game and its MDP, the GPU-batched simulator, the observation encoding, the policy network, the gated-allocation action space, the reward, and the self-play Elo league — everything the MCTS, GRPO, and PPO+GAE documents build on

Source of truth: the `orbit_wars_v12/` package

2026-06-19

Abstract

This is document 1 of four. It describes the parts of the v12 stack that are *the same* for every learner: the game itself, the batched simulator, what the policy sees (observation encoding), the network, what it emits (the gated-allocation action), the reward, and the self-play league that supplies opponents. The three sibling documents cover the learning algorithms that consume this foundation:

- `v12_ppo_gae.tex` — PPO with GAE (the shipped default) and the auxiliary reward-prediction head;
- `v12_grpo.tex` — the critic-free group-relative alternative and its (empirically ruled-out) advantage-estimator family;
- `v12_mcts.tex` — the AlphaZero-style PUCT proof of concept.

What exactly is shared differs by component: **the game, the simulator, the observation encoding, the network, and the action space are common to all three**. **The reward and the league are used by the two RL trainers (PPO+GAE and GRPO)**; the MCTS PoC instead scores leaves with a bounded heuristic and is not wired into the league. Every number here is read directly from `constants.py`, `config.py`, `env.py`, `worldgen.py`, `policy.py`, `distributions.py`, `rollout.py`, `league.py`, `bc.py`, and `checkpoint.py`.

Contents

1 The game and its MDP

Orbit Wars is a real-time-strategy game on a 100×100 board (`BOARD_SIZE`) with a sun of radius `SUN_RADIUS`$= 10$ fixed at the centre $(50, 50)$. *Planets* are owned by a player $(0 \dots N{-}1)$ or neutral (-1), hold an integer *garrison* of ships, and have a *production* rate (1–5 ships/tick added while owned). Planets within `ROTATION_RADIUS_LIMIT`$= 50$ of the centre *orbit* the sun at a per-world angular velocity $\omega \in [0.025, 0.05]$; the rest are static. A player launches *fleets* from owned planets; a fleet travels in a straight line at a ships-dependent speed, can be absorbed by the sun, and on reaching a planet fights for it.

Fleet speed and aiming. A fleet of n ships moves at

$$v(n) = \min\Big(1 + (\texttt{SHIP\_SPEED} - 1)\big(\tfrac{\log n}{\log 1000}\big)^{1.5},\ \texttt{SHIP\_SPEED}\Big), \qquad \texttt{SHIP\_SPEED} = 6.$$

Headings are computed analytically to *lead* an orbiting target (16 fixed-point iterations of the intercept solve, `LEAD_TARGET`), so neither the policy nor the scripts ever learn aiming — they choose a destination planet and the engine produces the heading.

Combat (official top-vs-second resolve). On a tick, all fleets that reach a planet are summed *per player* into arrivals $\texttt{arr} \in \mathbb{R}^{B \times E \times N}$. Let the two largest be $a_{(1)} \geq a_{(2)}$ with $a_{(1)}$ owned by player w. Survivors are $s = a_{(1)} - a_{(2)}$ (an exact tie annihilates: $s = 0$). If w already owns the planet it is reinforced $(+s)$; otherwise s is compared to the garrison and the planet *flips* to w when s exceeds it. This is the exact official rule and generalizes to any N.

Comets. Five comet events spawn on a hidden schedule (`COMET_SPAWN_STEPS`$= (50, 150, 250, 350, 450)$). Each event places 4 point-symmetric neutral comets that sweep an elliptical path (precomputed waypoints, `COMET_OFFICIAL`), carry a random garrison, produce 1 ship/tick once captured, and expire at the end of their path. Comets are deliberately excluded from capture/loss reward accounting (Section 7).

Winning. A side is alive while it owns a planet or has a fleet in flight. The game ends when one side remains, or at the horizon. In 2-player the outcome is $\text{sign}(s_0 - s_1)$ on total strength $s_p = \sum \text{garrisons} + \sum \text{fleet ships}$; 4-player uses either linear *placement* or official *winner-take-all* (Section 3).

MDP framing. From the acting seat's view (always re-indexed to ego = 0, below) the problem is a finite-horizon discrete-time MDP: state = the full board, action = a per-planet gated allocation (Section 6), transition = one `env_step` tick *given every other seat's simultaneous action*, reward = Section 7. Because all seats move simultaneously, the multi-agent game is only a stationary MDP once the opponents are fixed — which is exactly how each learner tames it: PPO/GRPO fix the opponents to frozen league members per rollout, and the MCTS PoC fixes them to a deterministic script.

2 The GPU-batched simulator (`GpuEnv`)

All of training runs inside `GpuEnv` (`env.py`), a fully tensorized re-implementation of the official engine. Every quantity is a `(B,*)` float32 tensor on `cfg.device`; one `env.step` advances all B parallel envs one tick with no host sync. The simulator is **bit-exact to the official Kaggle engine**, including the official elliptical comets (verified world-gen-exact and tick-exact; see the project memory `gpuenv-kaggle-parity` and `worldgen.py`, which mirrors `REFERENCE_orbit_wars.py` draw-order for draw-order).

State tensors. Planets: `p_alive`, `p_owner`, `p_x`, `p_y`, `p_radius`, `p_ships`, `p_prod`, `p_is_comet`, `p_init_x/y` (orbit anchor), `p_rotates`, `p_comet_vx/vy`, each `(B,Ec)` with `Ec=PLANET_CAP=48`. Fleets: `f_alive`, `f_owner`, `f_x`, `f_y`, `f_angle`, `f_ships`, `f_seq`, each `(B,Fc)` with `Fc=cfg.FLEET_CAP`. Per-env: `ang_vel`, `step_ct`, `done`; plus padded official-comet arrays `c_paths/c_len/c_ships/c_slot`.

One tick (`env_step`). In order: (1) spawn comets if this is a spawn step; (2) decode the ego action and every seat's action into launches (`_decode_target` for neural seats, `opponent_action` for scripted seats); (3) deduct launched ships from origins and append fleets into free slots; (4) add production to owned planets; (5) advance planet orbits / comet waypoints; (6) move fleets and resolve *swept* fleet–planet collisions against the planets' *moving* segments (a quadratic in the relative path), plus out-of-bounds and sun absorption; (7) resolve combat (top-vs-second, above); (8) apply new planet positions, clear removed fleets, increment `step_ct`. `env_step` returns per-env diagnostics (`captured`, `lost`, `captured_prod`, `lost_prod`, `launched_ships`, `owned_launch_ships`, ...) that the reward assembly consumes.

Determinism. With official comets (waypoint playback, no `torch.rand`) and deterministic seats, a tick is a pure function of state — the property the MCTS PoC relies on for snapshot/restore. The only stochastic seat is scripted code 0 (random heading) and the legacy non-official comet spawner.

3 Episodes, outcome, and seat-swap tiling

Horizon and termination. An episode runs `cfg.EPISODE_STEPS` ticks (= 500 full, 200 smoke). Per env, a rollout marks termination when `step_ct`$\geq T-2$, or #alive ≤ 1, or the ego seat is dead; finished envs are masked (no per-step CUDA-syncing early break).

Outcome scalar. At termination `_scores_outcome` freezes (a) the ego's pairwise score vs every seat (1 win / 0.5 draw / 0 loss) and (b) the scalar outcome $o \in [-1, 1]$:

$$o = \begin{cases} \operatorname{sign}\big(s_0 - \max_{p \geq 1} s_p\big) & \text{2p, or 4p } \texttt{OUTCOME\_4P="winner"}, \\ 2\,\overline{\text{pairwise}} - 1 & \text{4p } \texttt{OUTCOME\_4P="placement"} \text{ (beat-all} = +1). \end{cases}$$

In 2p, $o \in \{-1, 0, +1\}$ exactly.

Seat-swap world tiling (anti positional-overfit). Because the observation is in *absolute* board coordinates, an agent trained only as seat 0 overfits to seat-0 geometry (the edge inverts under a 180° board rotation; see memory `seat0-tempo-advantage`). The fix is deterministic, exhaustive seat coverage: `collect_ppo` tiles $W = B/P$ distinct worlds P-fold and stamps a per-block board rotation `env._rot_k`$\in \{0, 90, 180, 270\}°$ (2p uses $\{0, 180\}$, 4p uses $\{0, 90, 180, 270\}$). The board is 4-fold rotationally symmetric, so a $k \cdot 90°$ rotation about the centre is an *exact* seat remap; the fixed ego (pid 0) thus plays every seat's geometry in one batch. Evaluation/recalibration builds canonical (un-rotated) envs so ratings stay comparable.

4 Observation encoding (`env_encode`)

The encoder produces, for ego seat `ego`, four tensors: `entities` (B, E, F), `entity_mask` (B, E) (alive planets), `action_mask` (B, E) (legal sources), and `globals` (B, G). Everything is computed *ego-relative* so the same network plays any seat. Ownership is a **4-channel ego-relative seat one-hot** (v9): for a body with owner o, channel $r = (o - \text{ego}) \bmod n_{\text{players}}$ fires; channel 0 is always "me"; neutral is all-zero. In 2p only channels 0–1 ever fire, so 4p reuses the identical input space.

4.1 Fixed dimensions (`constants.py`)

Symbol	Value	Meaning
`PLANET_CAP` (E)	48	planet/comet slots per env = destination-categorical size
`F_DIM` (F)	194	per-planet feature width = 14 body + 180 threat
`N_BODY_FEATURES`	14	per-planet body features (4-channel seat one-hot ownership)
`N_THREAT_FLEETS`	30	top-30 inbound fleets/planet, selected by SIZE
`N_THREAT_FEATS`	6	per-fleet descriptor = 4-ch seat one-hot owner + eta + raw ships
`G_DIM` (G)	10	board-global features

4.2 Per-planet body features (14)

idx	expr	meaning (normalization)
0	`p_x / BOARD_SIZE`	x position $\in [0, 1]$
1	`p_y / BOARD_SIZE`	y position $\in [0, 1]$
2	`vmag / THREAT_MAX_SPEED`	orbital tangential speed $\lvert\omega\rvert \cdot d$ if rotating, else 0 (/6)
3	`cw`	rotation sense: +1 CCW, −1 CW, 0 if static/comet
4	`p_radius / 3`	planet radius
5	`p_prod / 5`	production rate
6	`p_ships`	garrison (RAW integer ship count; *un-normalized* — log compression removed)
7–10	seat one-hot	ownership [self, enemy+1, enemy+2, enemy+3]; neutral all-0
11	`dist / DIAG_HALF`	distance from centre (/ half-diagonal)
12	`comet`	1 if this slot is a comet
13	`actable`	1 if own ∧ alive ∧ ships> 0 (=`action_mask`)

4.3 Per-planet threat features (180 = 6 × 30)

For each planet, the `N_THREAT_FLEETS`= 30 **largest** inbound fleets *aimed at it* are selected by size ($\lvert\text{ships}\rvert \cdot 2^{20} - \text{seq}$, the −seq a stable tie-break); missing slots are zero-padded. Attribution uses the **apparent** target — the planet the fleet's ray points at, computed by `fleet_target_batch` *ignoring* the sun — so a fleet that will be **absorbed by the sun** before arriving still shows up in the threat list of the planet it is aimed at (the `etf` below is its time-to-apparent-target). Each descriptor (6 features):

- **owner one-hot (4)** — ego-relative seat one-hot of the fleet's owner (channel 0 self, 1–3 the opponents in seat order; empty slots all-zero);
- `etf` — ETA/`THREAT_ETA_SCALE` (/100);
- `slg` — |ships|, the **raw** inbound-fleet size (un-normalized; log compression removed).

`entities` = [body ‖ threat] is multiplied by `p_alive` so dead slots are 0.

v12 change (per-opponent threat identity). Pre-v12 the fleet/body owner was a single *sign* (+1 self / −1 any-opponent), collapsing all three 4p opponents into one value. The 4-channel one-hot lets the policy attribute an incoming fleet to a specific rival; it raised `F_DIM` from 104 to 194. Older 104-feature (or 101-feature scalar-owner) members still play via `checkpoint.LegacyObsAdapter`, which down-converts the current obs at forward time (sign = $\text{ch}_0 - \sum_{k=1}^{3} \text{ch}_k$). See memory `v12-threat-ownership-onehot`.

Motion-aware targeting and serving. The fleet→planet attribution and `etf` are **motion-aware**: `fleet_target_batch` solves the fleet-vs-*moving*-planet intercept time via a swept relative-velocity quadratic (the same form the engine's own collision uses), with each planet's velocity taken from a one-step finite difference $\mathbf{p} - \mathbf{p}_{\text{prev}}$ (tracked by the env; *internal* — never an obs feature). This leads orbiting planets and fast comets, so a fleet aimed at where a comet *will be* is attributed correctly (a static ray misses it); `etf` is the time-to-intercept in ticks. It reconstructs at serving with **no information loss**: the official Kaggle observation gives in-flight fleets as [id, owner, x, y, angle, from_pid, ships] and planet positions *every* turn, and the submission adapter

is a persistent B=1 `GpuEnv`, so the same finite-difference velocity is available there (the v12 adapter must track previous positions, matching planets by id). A per-source launch-ETA map fed to the WHERE head is a separate, not-yet-wired feature.

4.4 Board globals (10)

idx	expr	meaning
0	`step_ct / T`	game progress (time fraction)
1	`ang_vel * 10`	board rotation angular velocity
2	`ship_log_t(my_ships)`	my total garrison (log)
3	`ship_log_t(en_ships)`	all-opponents garrison, pooled (log)
4–6	`my_pl,en_pl,neu_pl / total`	fraction of live bodies mine / opponents' / neutral
7	`ship_log_t(my_fleet)`	my in-flight fleet ships (log)
8	`ship_log_t(en_fleet)`	opponents' in-flight fleet ships (log)
9	`min(npl,40)/40`	live-body count, scaled

The per-planet ownership is one-hot *per opponent*, but the global enemy aggregates (idx 3, 5, 8) pool *all* opponents (an everyone-vs-me summary).

Ship scaling (raw per-planet, log global). The *per-planet* ship features — the body garrison `b6` and the inbound-fleet sizes `slg` — are the **raw** integer counts; the `ship_log_t` compression on them was removed. The *global* ship aggregates (idx 2, 3, 7, 8) remain log-compressed via `ship_log_t`: they pool the whole board and would otherwise be large, un-normalized inputs. The reward potential Φ_{ship} (Section 7) likewise still uses `ship_log_t`. So the network now sees exact per-planet ship counts (where one ship can decide a capture) while the board-level summaries stay compressed.

5 The policy network (`PolicyNet`)

`build_policy(cfg)` instantiates an actor–critic that maps the four observation tensors to WHERE/GATE actor logits, a scalar value, and (optionally) an auxiliary reward prediction. The trunk maps `entities` (B, E, F) to per-planet tokens (B, E, h) with h = `HIDDEN` = 512; `cfg.ARCH` selects one of three interchangeable trunks, and everything downstream is identical.

5.1 Three trunks

Legacy ResNet-MLP (`ARCH="trunk"`, default):

```
entities -> proj: Linear(F -> 512)
         -> [optional] cross-planet single-head self-attention   (USE_ATTENTION, default OFF)
         -> [optional] GLU: x = x + glu_out(glu_val(x) * sigmoid(glu_gate(x)))  (USE_GLU, ON)
         -> N_RES_BLOCKS x  pre-LN ResNet-MLP: x = x + res_b(act(res_a(LN(x))))  (=86)
         -> ln_out: LayerNorm(512)
```

At the shipped defaults `USE_ATTENTION` is *False*, so the legacy trunk has *no cross-planet mixing inside the trunk* — planets interact only later, at the masked-mean value pool and the bilinear

WHERE head. `GRAD_CHECKPOINT` (default on) activation-checkpoints the residual blocks so the deep "512×86" stack fits in VRAM.

Stacked transformer (`ARCH="transformer"`):

```
entities -> proj: Linear(F -> 512)
         -> N_STEM_RES x pre-LN ResNet-MLP   (=1, pre-attention)
         -> N_TX_LAYERS x TxEncoderLayer     (=6 pre-LN MHA+MLP blocks)
         -> N_HEAD_RES x pre-LN ResNet-MLP   (=4, post-attention)
         -> ln_out: LayerNorm(512)
```

Each `TxEncoderLayer` is $x \leftarrow x + \mathrm{MHA}(\mathrm{LN}_1 x)$; $x \leftarrow x + \mathrm{MLP}(\mathrm{LN}_2 x)$, with `N_HEADS` heads (`scaled_dot_product_attention`, dead-planet keys masked) and MLP expansion `TX_MLP_RATIO`= 4. **Footgun:** `N_HEADS` must divide `HIDDEN`; the config default pairs 512 with 12 (invalid) — a 512-wide transformer must override `N_HEADS` (e.g. 8, head dim 64, as `INVITE_CFG` does).

Block-sequence (`ARCH="blockseq"`): an arbitrary ordered sequence of blocks parsed from `TRUNK_SPEC`, for hand-designed depth/attention layouts:

```
entities -> proj: Linear(F -> h)
         -> blocks from TRUNK_SPEC, in order:
              res  = pre-LN ResNet-MLP block
              attn = PURE cross-planet multi-head self-attention (pre-LN, residual, NO MLP)
              seq  = full transformer encoder layer (MHA + MLP, TX_MLP_RATIO)
         -> ln_out: LayerNorm(h)
```

e.g. `TRUNK_SPEC="res16,attn1,res64"`; `GRAD_CHECKPOINT` checkpoints every block. Three preset notebooks (`make_v12_nb.py -blockseq`) ship at $h = 256$, 8 heads, trained with `ALGO="mc"` (below) + the win-bet aux head: **(1)** `res16,attn1,res64` (81 blk, 11.3M); **(2)** `res16,attn1,res16,attn1,res32` (66 blk, 9.5M); **(3)** `res8,attn1,seq16,res32` (8 res → attn → 16 encoder layers → 32 res, 57 blk, 18.7M). Each notebook carries an *MCTS depth/breadth estimate* cell (measured B=1 2-core forward → sims/turn at the ∼1 s budget).

Critic-free Monte-Carlo policy gradient (`ALGO="mc"`). A third optimizer alongside PPO+GAE and GRPO: near-vanilla PPO with **no critic and no GAE** — the advantage is the *discounted Monte-Carlo return-to-go* $A_t = G_t = \sum_{k \geq t} \gamma^{k-t} r_k$ (reset at episode end), batch-whitened (center + global scale). It reuses the GRPO clipped-surrogate update (no value loss), and `VALUE_WARMUP_ITERS` is forced off. Computed in `collect_ppo`; dispatched in `train.py`.

5.2 Heads (shared by all trunks)

Per planet the head input concatenates the trunk token with a board-globals embedding `gp` = relu(`g_embed`(`globals`)) $\in \mathbb{R}^{d_g}$, $d_g =$ `D_G` $= 32$, giving $hh \in \mathbb{R}^{h+d_g} = \mathbb{R}^{544}$.

- **WHERE** (`DEST_HEAD="bilinear"`, default): pointer scores `dest_logits`$[s, d] = \langle$`dest_q`(hh_s), `dest_k`(hh_d) a (B, E, E) source×destination matrix.
- **GATE:** `gate_head` Linear($544 \to 1$) ⇒ per-source launch logit (B, E).
- **VALUE (critic):** masked-mean pool of trunk tokens over live planets, concat `gp`, `val_in`→`VALUE_RES_BLOCK` 4 pre-LN ResNet-MLP blocks→`val_out`. Targets are **PopArt**-normalized (output-preserving running mean/var; the head predicts normalized values, raw V = denorm). The critic is used by PPO; GRPO leaves it untrained (unless `GRPO_AUX_VALUE`); the MCTS `value="net"` mode reads it.

- **AUX reward** (optional, `AUX_REWARD_PRED`, default off): a shallow `rew_in`→act→`rew_out` off the same pooled features (document 2). Built only when enabled, so checkpoints are byte-identical when off.

5.3 Parameter counts (instantiated, fp32)

$E = 48$, $G = 10$, $d_g = 32$, `VALUE_RES_BLOCKS`= 4; heads = g_embed+WHERE+gate+value (aux off).

config	trunk	heads	**total**	fp32 / fp16
512×86 legacy (default)	46.16 M	2.94 M	**49.10 M**	196 / 98 MB
512×32 legacy	17.73 M	2.94 M	**20.68 M**	83 / 41 MB
256×32 legacy	4.64 M	0.59 M	**5.23 M**	21 / 10 MB
512h / 6L / 8-head transformer	21.65 M	2.94 M	**24.59 M**	98 / 49 MB
256h / 4L / 8-head (`INVITE_CFG`)	3.62 M	0.34 M	**3.96 M**	16 / 8 MB

The Kaggle 100 MB cap implies ≤ 26 M params at fp32: the default 512×86 ships only at fp16; the 512×32 legacy and 512h/6L transformer are the largest fp32-shippable trunks.

6 Action space & distributions

The actor is a **gated per-source allocation**. For every owned source planet it emits a WHERE distribution over destinations and a launch GATE. *On fire, the planet dispatches its full garrison routed by the WHERE row; on no-fire it holds.* The action bundle is $(B, E, E{+}1)$: $[:E]$ the allocation row, $[E]$ fire $\in \{0, 1\}$. Log-prob and entropy *sum over owned sources*; the WHERE term only counts when firing.

Masking. WHERE logits are clamped to $\pm$`LOGIT_CLAMP` $= 8$ then masked to -10^9 on: the diagonal (self), dead destinations (`alive`< 0.5), and sun-blocked destinations (`reach`< 0.5 — a straight shot absorbed by the sun, recovered geometrically by `compute_reach` identically in rollout and update). The support is exactly the legal, reachable targets.

Gate. $g = \sigma($`gate_logit`$)$. **Train:** $p = g$ (smooth, full-support gradient — the deploy deadzone clamp used to zero it and vanish the gate gradient). **Deploy:** hard deadzone $p = \text{clip}((g -$ `GATE_TRIM_LO`$)/($`GATE_TRIM_HI`$-$`GATE_TRIM_LO`$), 0, 1)$. Then $p \leftarrow \text{clip}(p,$ `GATE_EPS`$, 1-$`GATE_EPS`$)$, fire $\sim$ Bernoulli(p).

`GatedCatDist` (categorical WHERE, `WHERE_DIST="categorical"`, default). After masking, `lsm` $= \log\text{softmax}($`glogits`$)$. *Sample:* dest $\sim$ Categorical, fire $\sim$ Bernoulli. *Log-prob (per source):* $\ell = [\text{fire} \log p + (1-\text{fire}) \log(1-p)] + \text{fire} \cdot$ `lsm`[dest], summed over owned sources. *Entropy:* Bernoulli gate entropy $+$`ENT_DIR_COEF`$\cdot p \cdot H_{\text{cat}}$ with $H_{\text{cat}} \leq \ln E$; `ENT_DIR_COEF`$= 0$ by default, so the bonus is *gate-only*. The bounded-sensitivity log softmax log-prob is what makes PPO-after-BC stable.

`GatedAllocDist` (Dirichlet WHERE, v6 A/B). $\alpha = \text{softplus(logits)} \cdot$ `ALLOC_KAPPA` $+ 10^{-3}$; rows $\sim$ Dirichlet, splitting the garrison across destinations. Its greedy decode uses a temperature-sharpened softmax ($\tau =$ `GREEDY_TAU` $= 0.5$) because the Dirichlet mean spreads the garrison too thin to land a concentrated strike.

Decode (`_decode_target`). A fired source's full garrison S is routed by its renormalised off-diagonal WHERE row $\hat{A}$: $n_{s,d} = \lfloor \hat{A}_{s,d}\, S_s \rfloor$ to alive, legal destinations, kept only if $n \geq$ `MIN_LAUNCH_SHIPS` $= 2$ (dribbles stay home — a structural budget, no penalty). Each launch is aimed at the orbiting intercept. So although the categorical WHERE samples one destination, the *decode* is a deterministic full-garrison strike to that target.

7 Reward (used by the RL trainers)

The reward is assembled inside `collect_ppo` (`rollout.py`) and consumed by PPO+GAE and GRPO; the MCTS PoC does not use it. Two per-step regimes (selected by `USE_POTENTIAL_SHAPING`, default *off* — the small legacy dense channels, so the terminal W/L dominates) feed a per-step r_t, plus a sparse terminal payment; everything is divided by `PPO_REWARD_SCALE`$= 100$ before it reaches the learner.

7.1 Optional: potential-based shaping

Policy-invariant shaping (Ng et al.) on two potentials, evaluated by `_potentials` as the ego's log-ship margin and production share vs the *strongest* opponent:

$$\Phi_{\text{ship}} = \texttt{ship\_log\_t}(s_0) - \texttt{ship\_log\_t}\big(\max_{p\geq 1} s_p\big), \qquad \Phi_{\text{prod}} = \frac{P_0 - \max_{p\geq 1} P_p}{\sum P}.$$

Per step (for still-active envs),

$$r_t = \texttt{SHAPE\_SHIP}\,(\gamma\Phi'_{\text{ship}} - \Phi_{\text{ship}}) + \texttt{SHAPE\_PROD}\,(\gamma\Phi'_{\text{prod}} - \Phi_{\text{prod}}), \qquad \texttt{SHAPE\_SHIP} = \texttt{SHAPE\_PROD} = 25,\ \gamma = \texttt{GAMMA} =$$

where primes are the post-step potentials. Being a γ-difference of a potential, this shaping is policy-invariant (it cannot change the optimal policy); it only densifies credit.

7.2 Default: legacy dense channels

With `USE_POTENTIAL_SHAPING=False` (the default), r_t is the sum of three measured channels:

- **capture:** `CAPTURE_REWARD`$\big[$(captured+`CAPTURE_PROD_SCALE`$\,$captured_prod)$-$`CAPTURE_LOSS_FRAC`(lost+ `CAPTURE_PROD_SCALE`$\,$lost_prod)$\big]$, `CAPTURE_REWARD` $= 30$, comet flips excluded;
- **production milestone:** +`PROD_MILESTONE_REWARD` $= 20$ each time total strength doubles past `PROD_MILESTONE_BASE`$= 100$ (monotone — losing ships never claws it back);
- **launch shaping** (first `LAUNCH_WINDOW`$= 150$ steps): a small per-ship-launched bonus, per-step- and per-game-capped (`LAUNCH_STEP_CAP`, `LAUNCH_GAME_CAP`); unspent budget is cashed out on a win.

7.3 Terminal outcome payment

Both regimes add the sparse game result to the *last active step*. With episode length ℓ and len_eff $= (\ell - \texttt{DECAY\_START\_STEP})_+$, `DECAY_START_STEP` $= 100$,

$$O = \begin{cases} o \cdot \texttt{WIN\_DECAY}^{\,\text{len\_eff}} \cdot \texttt{WIN\_BONUS}, & o > 0, \\ o \cdot \texttt{LOSS\_DECAY}^{\,\text{len\_eff}} \cdot \texttt{LOSS\_PENALTY}, & o \leq 0, \end{cases} \qquad \texttt{WIN\_BONUS} = \texttt{LOSS\_PENALTY} = 1000,$$

added as O/`PPO_REWARD_SCALE`. In 4p placement mode the fractional o scales the magnitudes linearly; in 2p $o \in \{-1, 0, +1\}$.

Outcome decay. The current defaults are `WIN_DECAY` = 1.0 and `LOSS_DECAY` = 0.9995: an asymmetric "lose-slower" shaping (a late loss is penalized slightly less than an early one — a mild survival incentive), with wins undecayed. The provenance doc `set-ups/1.md` used a 0.995 symmetric decay past step 100. This is purely a reward-shaping knob: an earlier write-up blamed an asymmetric decay for the GRPO passivity collapse, but that did not hold up — the collapse is caused by the advantage-estimator changes (document 3), not by the decay.

8 Self-play league (the curriculum)

There are *no hand-coded curriculum stages*: the dual-Elo PFSP league (`league.py`) *is* the curriculum. Each iteration samples an opponent (or a 4p trio), the learner rolls out against it, and the result updates ratings. The two RL trainers share this verbatim; the MCTS PoC does not use it.

Members and ratings. Every member carries a 2p rating `elo` and a 4p rating `elo4`. Fixed scripted *anchors* seed the league — `random`, `starter`, `greedy`, `medium`, `intermediate` — at pinned ratings (e.g. `ELO_STARTER`= 1350, `ELO_MEDIUM`= 1560). Frozen learner *snapshots* (added every `SELFPLAY_REFRESH` iters) and *invited* external agents fill the rest. The learner itself is grounded on a **rolling self-anchor** — its own frozen high-water snapshot — so its rating never clamps on saturated scripts (memory `v9-elo-saturation-confirmed`).

Scripted opponents (`opponent_action`). The anchor "kinds" map to deterministic (except code 0) rules of escalating strength:

code	kind	behaviour (one launch per owned planet, half garrison ≥ 20)
0	random	random heading
1	starter	fire at the nearest *static* non-owned planet
2	noop	never launches
3	medium	nearest non-owned planet incl. orbiting, lead-aimed at the intercept
4	greedy	nearest beatable non-owned planet within `CAPTURE_RADIUS`, just-enough ships
5	intermediate	in-flight-aware capture from the nearest capable source + counter + rebalance + comet-escape

PFSP matchmaking. Per-member sampling weight (`_weights`) is a PFSP function of the expected score p (`PFSP_MODE="even"`: $p(1-p)$ — evenly-matched opponents) times a behavioural-footprint mix (2p), invited/scripted boosts, and an advance lock (`greedy`/`medium` stay locked until the learner clears `ADVANCE_TRIGGER_ELO`). A scripted-share cap keeps scripts collectively $\leq$ `SCRIPT_SHARE_CAP` = 0.30 while any neural member carries weight; a 2p-mastered script leaves 2p matchmaking.

Dual Elo and the 2p/4p mix. 2p results are a single Elo update; 4p FFA results are decomposed into pairwise updates at `ELO_K4`/k each, cross-coupled into the 2p rating by `ELO_COUPLING`= 0.25. The fraction of 4p iterations is `share_4p` from an EMA of the (elo2−elo4) gap, *pinned* to 0.25 at the shipped defaults (`S4_MIN` = `S4_MAX` = 0.25, a 3:1 2p:4p ratio; memory `share4p-pinned-1to1`).

Recalibration and measured eviction. Periodic recals re-ground both rating scales on the top self-anchor and blend into the live ratings (first recal is raw). A **measured eviction** (`evict_mastered`) removes any league member the learner has mastered under one standard — 2p seat-averaged score $>$ `MASTER_EVICT_2P_WR` $= 0.90$ *or* 4p 1st-place rate vs three copies $>$ `MASTER_EVICT_4P_WR` $= 0.75$ — covering scripted anchors and neural players alike, but never the self-anchors, the newest snapshot, or the `random`/`greedy` watchdogs. Dormant *watchdogs* (`random`, `greedy`) re-enter as live opponents if the learner underperforms its own rating prediction by `WATCHDOG_TOL` — a collapse alarm. See memory `v12-clean-league-measured-eviction` and `v12-eviction-league-players`.

Best-checkpoint selection. Checkpoints are scored on a held-out world pool by a *mixed gauntlet*: a 2p score vs (`starter`, `intermediate`, `greedy`) blended with a 4p FFA score (scripted trio + invited-neural trio), weighted by `GAUNTLET_4P_W`. The held-out seeds are never touched by training, so the score measures generalization and stays comparable across the run.

8.1 Behaviour-cloning warm-start (`bc_pretrain`)

From-scratch RL reliably falls into a passivity trap (memory `orbit-wars-policy-failure-diagnosis`), so training optionally starts from a BC clone of the `medium` rule. The clone visits its own states against a mixed scripted-opponent schedule and minimizes BCE(gate) + CE(masked WHERE) on firing sources, expressing the expert as one-hot WHERE rows + fire. This produces a committing, aiming policy (memory `bc-warmstart-gated-alloc`); both RL trainers then resume from it.

9 How each algorithm plugs in

component	who uses it
game + `GpuEnv` + episodes	all three (PPO+GAE, GRPO, MCTS)
observation encoding + network	all three
gated-allocation action + decode	all three
reward (Section 7)	PPO+GAE, GRPO (MCTS uses a heuristic leaf value instead)
self-play league (Section 8)	PPO+GAE, GRPO (MCTS PoC plays a fixed scripted seat)
BC warm-start	PPO+GAE, GRPO (and the planned MCTS self-play loop)

The learner's per-iteration loop — PFSP-sample opponents, `collect_ppo` rollout, `ppo_update` or `grpo_update`, Elo update, periodic recal/eviction, best-ckpt gauntlet — lives in `train.py::Trainer.train` and is identical for PPO and GRPO save for the update call and the advantage construction. Those two updates, and the MCTS search, are the subjects of the three sibling documents.